

Лабораторна робота №7

Автентифікація та авторизація

Мета: Метою лабораторної роботи є набуття практичних навичок реалізації автентифікації та авторизації у Node.js-застосунках шляхом розширення застосунку з лабораторної роботи №6

Завдання №1. Реєстрація користувачів з хешуванням пароля

Встановіть пакет bcryptjs

```
Run `npm audit` for details.  
PS D:\Homework\3 курс\node github> npm install --save-dev @types/bcryptjs @types/jsonwebtoken @types/cookie-parser  
added 4 packages, and audited 488 packages in 689ms
```

Рис. 1 – Встановлення пакетів

Завдання №2. Логін з access та refresh токенами

Встановіть пакети jsonwebtoken та @types/jsonwebtoken. Додайте змінну JWT_SECRET у .env для локальної розробки та задайте її через fly secrets set для сервера.

Створіть Mongoose-модель User з полями для електронної пошти, хешу пароля та дати створення. Додайте унікальний індекс на полі електронної пошти, щоб БД не дозволяла дублікати.

```
server.ts  database.ts  .env  .env.example  fly.toml  
1 MONGODB_URI=mongodb+srv://<user>:<password>@cluster0.jgpyvoz.mongodb.net/lab4db  
2 PORT=3000  
3 JWT_SECRET=super_secret_access_key  
4 JWT_REFRESH_SECRET=super_secret_refresh_key
```

Рис. 2 – Оновлення .env

Також відповідно оновлено github secrets

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000						
Змн.	Арк.	№ докум.	Підпис	Дата							
Розроб.		Жеребцов Д.В.			Звіт з лабораторної роботи №5			Лім.	Арк.	Аркуші	
Перевір.		Оринчак А.І								1	10
Реценз.								ФІКТ, гр. ІПЗк-24-1			
Н. Контр.											
Зав.каф.											

```

src > models > ts user.models.ts > UserSchema > email
1  import mongoose, { Schema, Document } from "mongoose";
2  import bcrypt from "bcryptjs";
3
4  export interface IUser extends Document {
5      email: string;
6      password: string;
7      createdAt: Date;
8      updatedAt: Date;
9  }
10
11  const UserSchema = new Schema<IUser>({
12      {
13          email: {
14              type: String,
15              required: [true, "Email є обов'язковим"],
16              unique: true,
17              lowercase: true,
18              trim: true,
19          },
20          password: {
21              type: String,
22              required: [true, "Пароль є обов'язковим"],
23          },
24      },
25      { timestamps: true },
26  });
27

```

Рис. 3 – Створення mongoose-моделі user

Реалізуйте ендпоінт POST /auth/register, що приймає електронну пошту та пароль. Виконайте Zod-валідацію вхідних даних: коректний формат пошти та мінімальна довжина пароля.

Лістинг:

```

const authSchema = z.object({
  email: z.string().email("Некоректний формат email"),
  password: z.string().min(6, "Пароль має містити щонайменше 6 символів"),
});

router.post("/register", async (req: Request, res: Response, next: NextFunction):
Promise<void> => {
  try {
    const { email, password } = authSchema.parse(req.body);

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      res.status(409).json({ message: "Користувач з таким email вже існує" });
      return;
    }

    const newUser = await User.create({ email, password });

    res.status(201).json({
      id: newUser._id,
      email: newUser.email,
      createdAt: newUser.createdAt,
    });
  } catch (error) {
    next(error);
  }
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

Винесіть хешування у pre-save hook моделі: він має спрацьовувати лише якщо поле пароля було змінено.

Лістинг:

```
UserSchema.pre("save", async function () {
  if (!this.isModified("password")) {
    return;
  }

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
```

При спробі зареєструвати вже наявну електронну пошту повертайте 409 Conflict із зрозумілим повідомленням

Лістинг:

```
const existingUser = await User.findOne({ email });
if (existingUser) {
  res.status(409).json({ message: "Користувач з таким email вже існує" });
  return;
}
```

Відповідь успішної реєстрації не повинна містити хеш пароля.

Лістинг:

```
res.status(201).json({
  id: newUser._id,
  email: newUser.email,
  createdAt: newUser.createdAt,
});
```

Покрийте ендпоінт інтеграційними тестами (Jest + Supertest + mongodbmemory-server за зразком лаб. 4).

```
PASS tests/auth.test.ts
Інтеграційні тести API Auth
  POST /api/auth/register
    ✓ Повинен успішно зареєструвати користувача (88 ms)
    ✓ Повинен повернути 409 Conflict при дублюванні email (53 ms)
  POST /api/auth/login
    ✓ Повинен успішно залогінити та видати cookies з токенами (97 ms)
    ✓ Повинен повернути 401 при неправильному паролі (102 ms)
  POST /api/auth/logout
    ✓ Повинен очистити cookies при виході (4 ms)
```

Рис. 5 – Покриття тестами

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання №3. Захищені маршрути та авторизація власника

Реалізуйте middleware `requireAuth`: зчитує `access_token` з `cookie`, верифікує підпис та строк дії, у разі успіху прикріплює інформацію про користувача (принаймні `userId`) до `req` (через розширення типу `Request`). У разі невдачі повертає `401 Unauthorized` без подробиць у тілі відповіді.

Лістинг `requireAuth`:

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";

declare global {
  namespace Express {
    interface Request {
      userId?: string;
    }
  }
}

export const requireAuth = (req: Request, res: Response, next: NextFunction): void => {
  const token = req.cookies?.access_token;

  if (!token) {
    res.status(401).json({ message: "Unauthorized" });
    return;
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET as string) as {
      userId: string;
    };
    req.userId = decoded.userId;
    next();
  } catch (error) {
    res.status(401).json({ message: "Unauthorized" });
  }
};
```

Застосуйте `requireAuth` до маршрутів, що змінюють стан: `POST`, `PUT`, `DELETE` для ресурсу з лаб. 3–4. Маршрути читання (`GET`) можуть залишатись публічними або теж стати захищеними — на ваш вибір, але рішення зафіксуйте у коді послідовно.

Застосування `requireAuth`:

```
router.post(
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

```
"/", requireAuth, validate(createRecipeSchema), async (req: Request, res: Response, next: NextFunction) => {
```

```
router.put(
  "/:id",
  requireAuth,
  validate(updateRecipeSchema),
  async (req: Request, res: Response, next: NextFunction): Promise<void> => {
```

```
router.delete(
  "/:id", requireAuth, async (req: Request, res: Response, next: NextFunction):
  Promise<void> => {
```

Додайте у Mongoose-схему ресурсу поле ownerId (ObjectId з посиланням на колекцію користувачів). При створенні запису через POST заповнюйте це поле з req.userId.

Оновлення mongoose-схеми ресурсу:

```
export interface IRecipe {
  id: string; // поле id ще було передбачено в 3 лабі, тому залишаю як є
  title: string;
  description?: string;
  difficulty: "easy" | "medium" | "hard";
  cookingTimeMinutes: number;
  ingredients: string[];
  ownerId: mongoose.Types.ObjectId | string;
}

const recipeSchema = new Schema<IRecipeDocument>(
  {
    // інші поля
  },
  {
    ownerId: {
      type: Schema.Types.ObjectId,
      ref: 'User',
      required: [true, 'Власник є обов'язковим']
    }
  }
)
```

У PUT та DELETE перед застосуванням змін перевіряйте, що ownerId запису збігається з req.userId. У разі невідповідності повертайте 403 Forbidden

Додаємо перевірку на власність для PUT та DELETE запитів:

```
if (recipe.ownerId.toString() !== req.userId) {
  res.status(403).json({ message: "Forbidden" });
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    return;
}

```

Покрийте інтеграційними тестами

PASS tests/recipe.test.ts (5.774 s)

Unit-тести Recipe

- ✓ Повинна успішно створювати документ із правильними дефолтами та віртуальними полями (96 ms)
- ✓ Повинна викидати помилку при невалідних даних (кастомні валідатори Mongoose) (90 ms)
- ✓ Повинна коректно форматовувати час (години та хвилини) (93 ms)

Інтеграційні тести API Recipe

POST /api/recipes

- ✓ Повинен створити новий рецепт та повернути статус 201 (115 ms)
- ✓ Повинен повернути 400 при помилці валідації Zod (немає назви) (119 ms)
- ✓ Повинен повернути 401, якщо запит без токена (93 ms)

GET /api/recipes

- ✓ Повинен повертати об'єкт з data та pagination (104 ms)
- ✓ Повинен коректно застосовувати фільтрацію Mongoose (92 ms)
- ✓ Повинен фільтрувати за title та maxTime (101 ms)

GET /api/recipes/random

- ✓ Повинен повертати випадковий рецепт (93 ms)
- ✓ Повинен повертати 404, якщо база порожня (94 ms)

GET /api/recipes/:id

- ✓ Повинен повертати рецепт за існуючим UUID (98 ms)
- ✓ Повинен повертати 404, якщо рецепт з валідним ID не знайдено (93 ms)

PUT /api/recipes/:id

- ✓ Повинен оновлювати існуючий документ (власник) (101 ms)
- ✓ Повинен повернути 403 Forbidden, якщо оновлює не власник (91 ms)
- ✓ Повинен повернути 404, якщо рецепта не існує (90 ms)

DELETE /api/recipes/:id

- ✓ Повинен видаляти існуючий документ (власник) (96 ms)

Рис. 6 – Покриття тестами

Приклад оновлення тестів до запитів, які вимагають requireAuth:

```

describe("POST /api/recipes", () => {
  it("Повинен створити новий рецепт та повернути статус 201", async () => {
    const res = await request(app)
      .post("/api/recipes")
      .set("Cookie", [`access_token=${testToken}`])
      .send({
        title: "Млинці",
        difficulty: "easy",
        cookingTimeMinutes: 20,
        ingredients: ["Молоко", "Яйця"],
      });

    expect(res.status).toBe(201);
    expect(res.body.title).toBe("Млинці");
    expect(res.body.id).toBeDefined();
    expect(res.body.ownerId).toBe(testUserId);
  });

  it("Повинен повернути 400 при помилці валідації Zod (немає назви)", async ()
=> {

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

const res = await request(app)
  .post("/api/recipes")
  .set("Cookie", [`access_token=${testToken}`])
  .send({
    difficulty: "easy",
    cookingTimeMinutes: 20,
    ingredients: ["Молоко"],
  });

expect(res.status).toBe(400);
expect(res.body.message).toContain("Zod");
});

it("Повинен повернути 401, якщо запит без токена", async () => {
  const res = await request(app)
    .post("/api/recipes")
    .send({
      title: "Млинці",
      difficulty: "easy",
      cookingTimeMinutes: 20,
      ingredients: ["Молоко"],
    });

  expect(res.status).toBe(401);
});
});

```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	98.83	88.73	100	98.75	
src	100	100	100	100	
app.ts	100	100	100	100	
src/middleware	97.05	92.85	100	96.77	
errorHandler.ts	100	91.66	100	100	43
requireAuth.ts	91.66	100	100	90.9	25
validate.ts	100	100	100	100	
src/models	96.42	87.5	100	96.29	
recipe.model.ts	100	100	100	100	
user.model.ts	88.88	50	100	88.88	30
src/routes	100	100	100	100	
auth.ts	100	100	100	100	
recipe.ts	100	100	100	100	
src/schemas	100	100	100	100	
recipe.schema.ts	100	100	100	100	
src/storage	100	72.72	100	100	
recipe.ts	100	72.72	100	100	16-18
tests	95.45	50	100	94.73	
setup.ts	95.45	50	100	94.73	8
Test Suites: 2 passed, 2 total Tests: 42 passed, 42 total Snapshots: 0 total Time: 5.186 s, estimated 6 s Ran all test suites.					
PS D:\HomeWork\3 курс\node github>					

Рис. 7 – Загальне покриття тестами

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

Тестування:

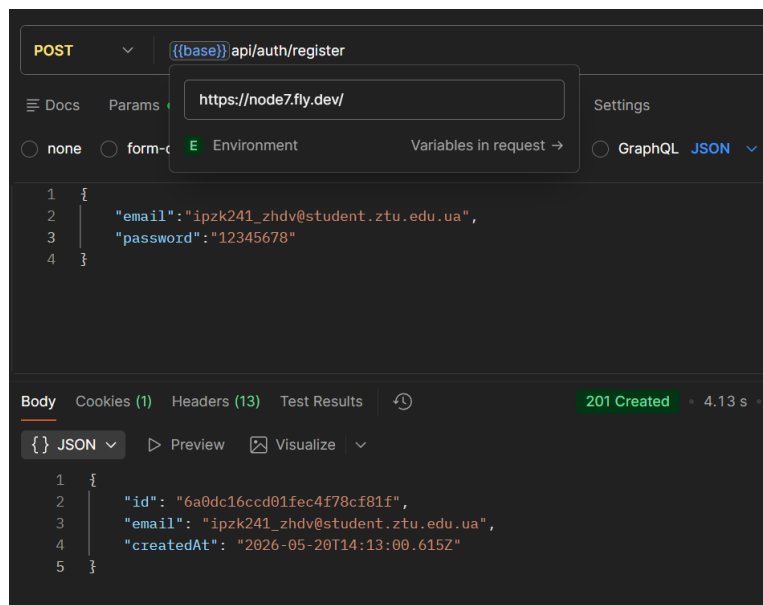


Рис. 8 – Реєстрація

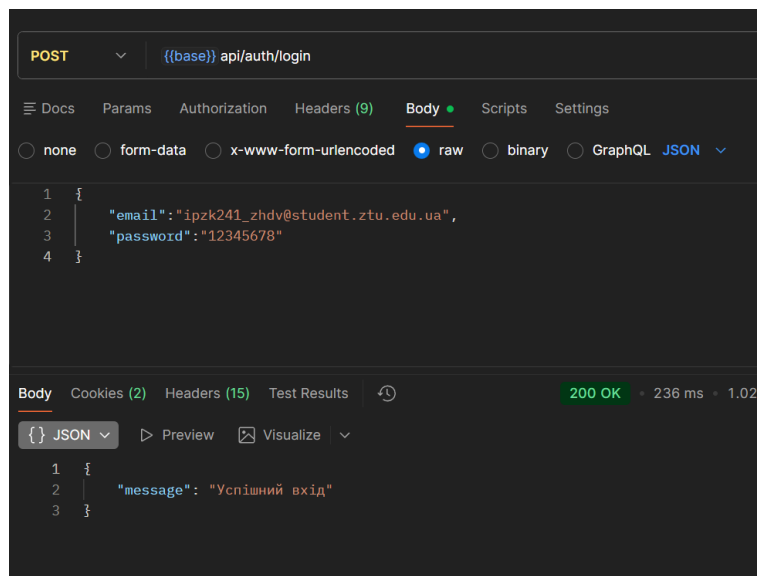


Рис. 9 – Авторизація

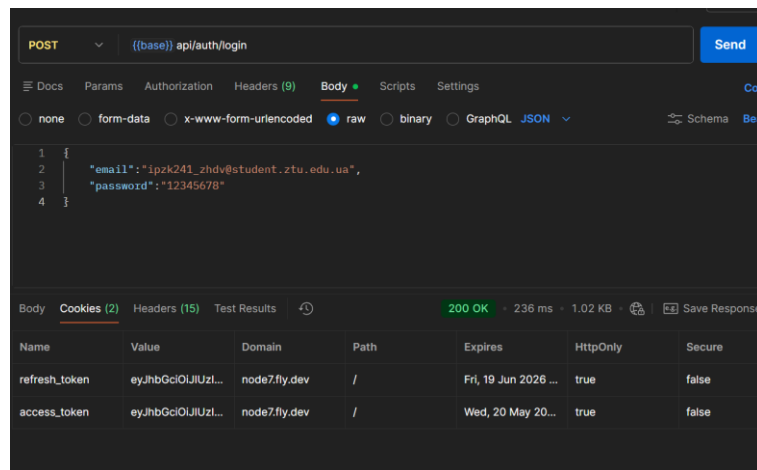


Рис. 10 – Отримання jwt у cookie

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.2.000	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		

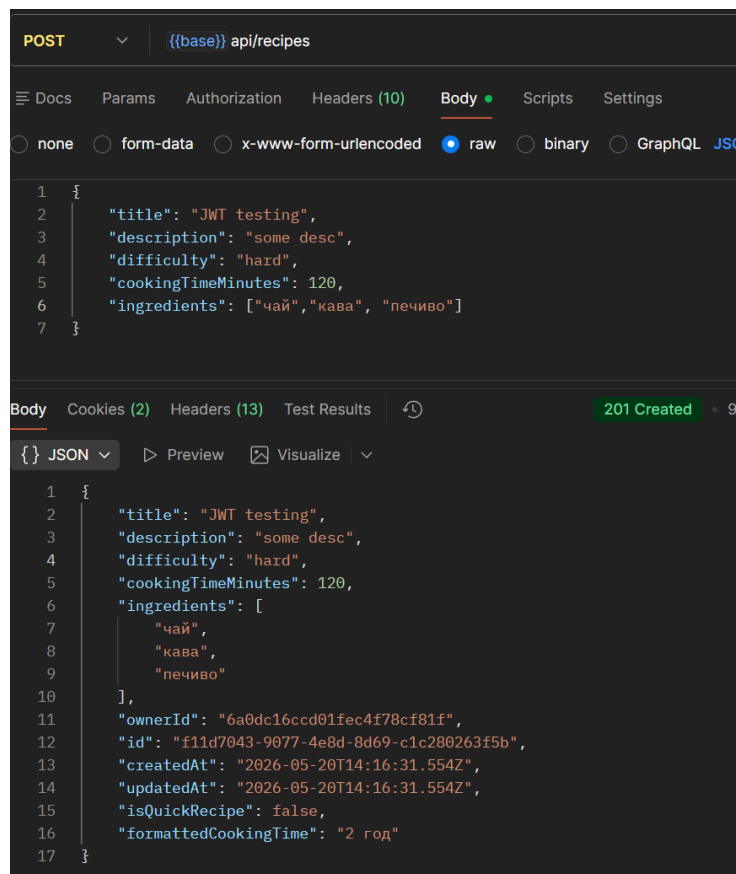


Рис. 11 – post

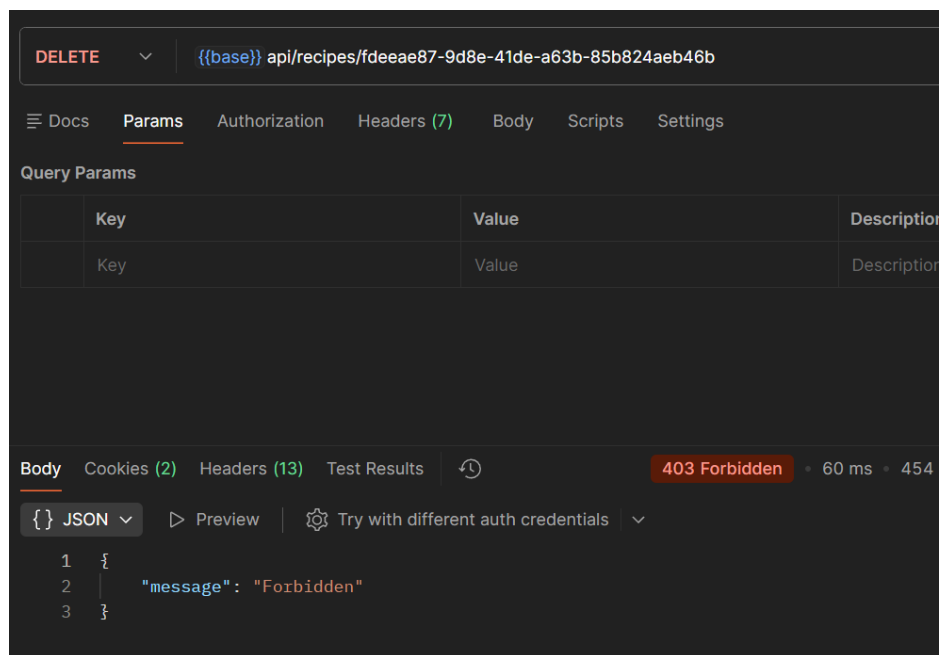


Рис. 12 – Спроба видалити чужий рецепт

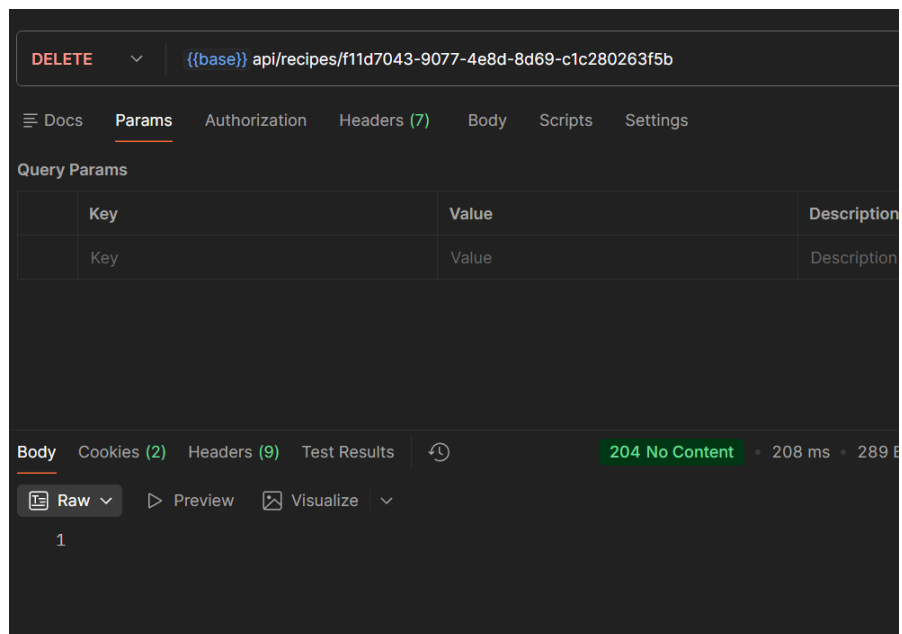


Рис. 13 – Видалення власного рецепту

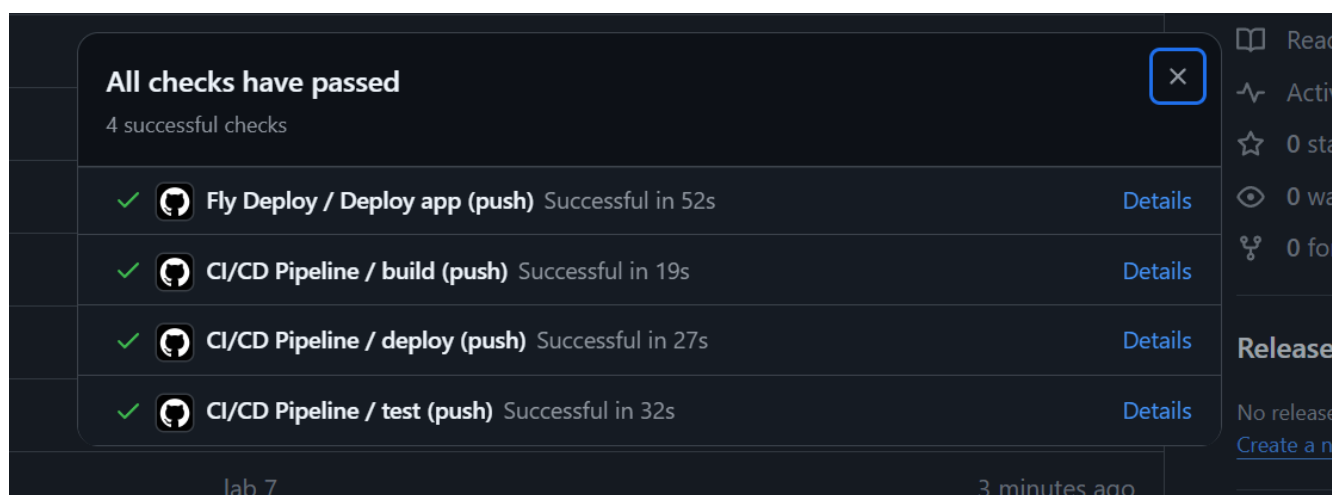


Рис. 14 – ci/cd, deploy, fly.io workflows на гітхабі

Висновки: набуто теоретичних та практичних навичок з реалізації автентифікації та авторизації у Node.js-застосунках.